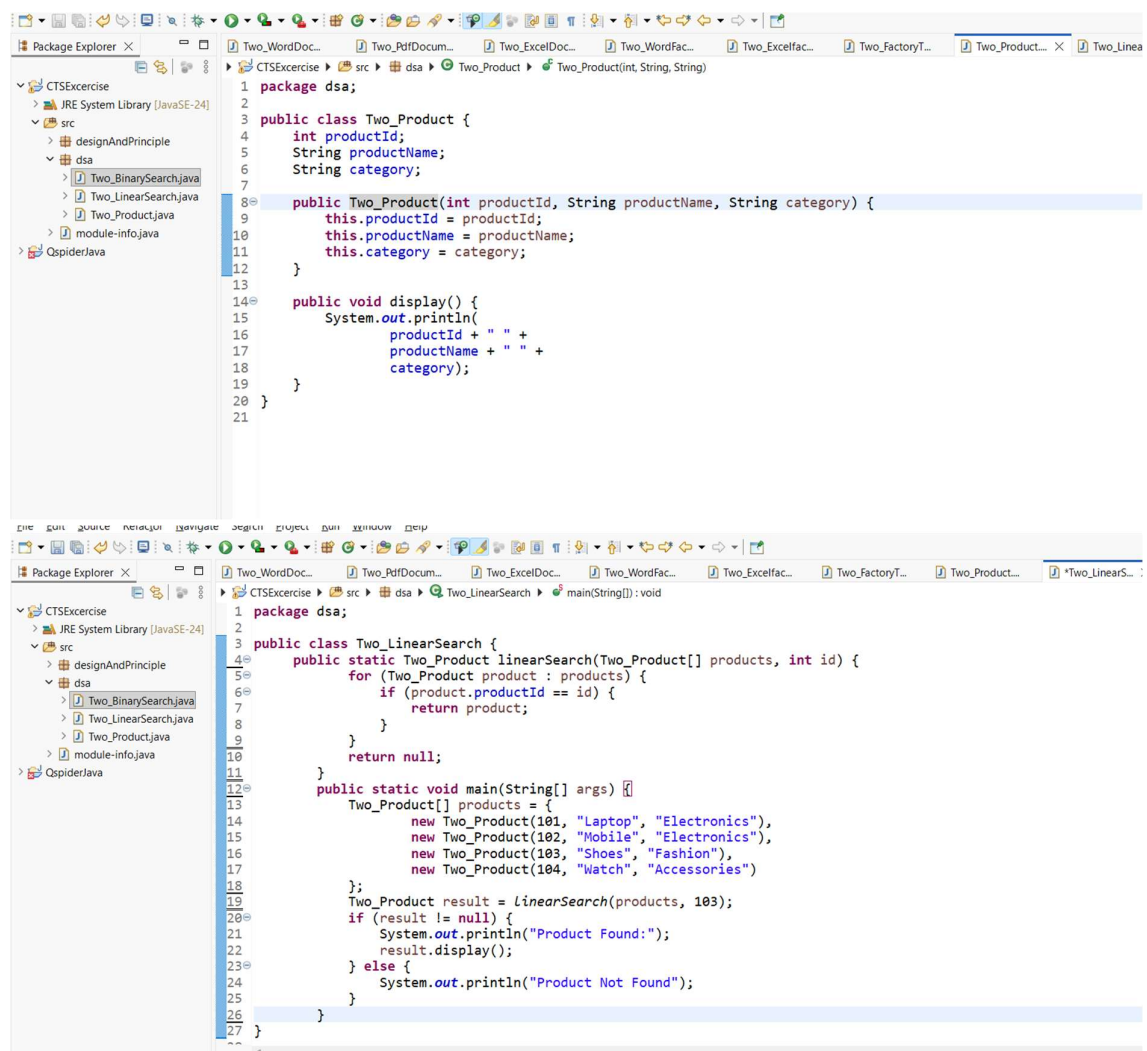


DSA

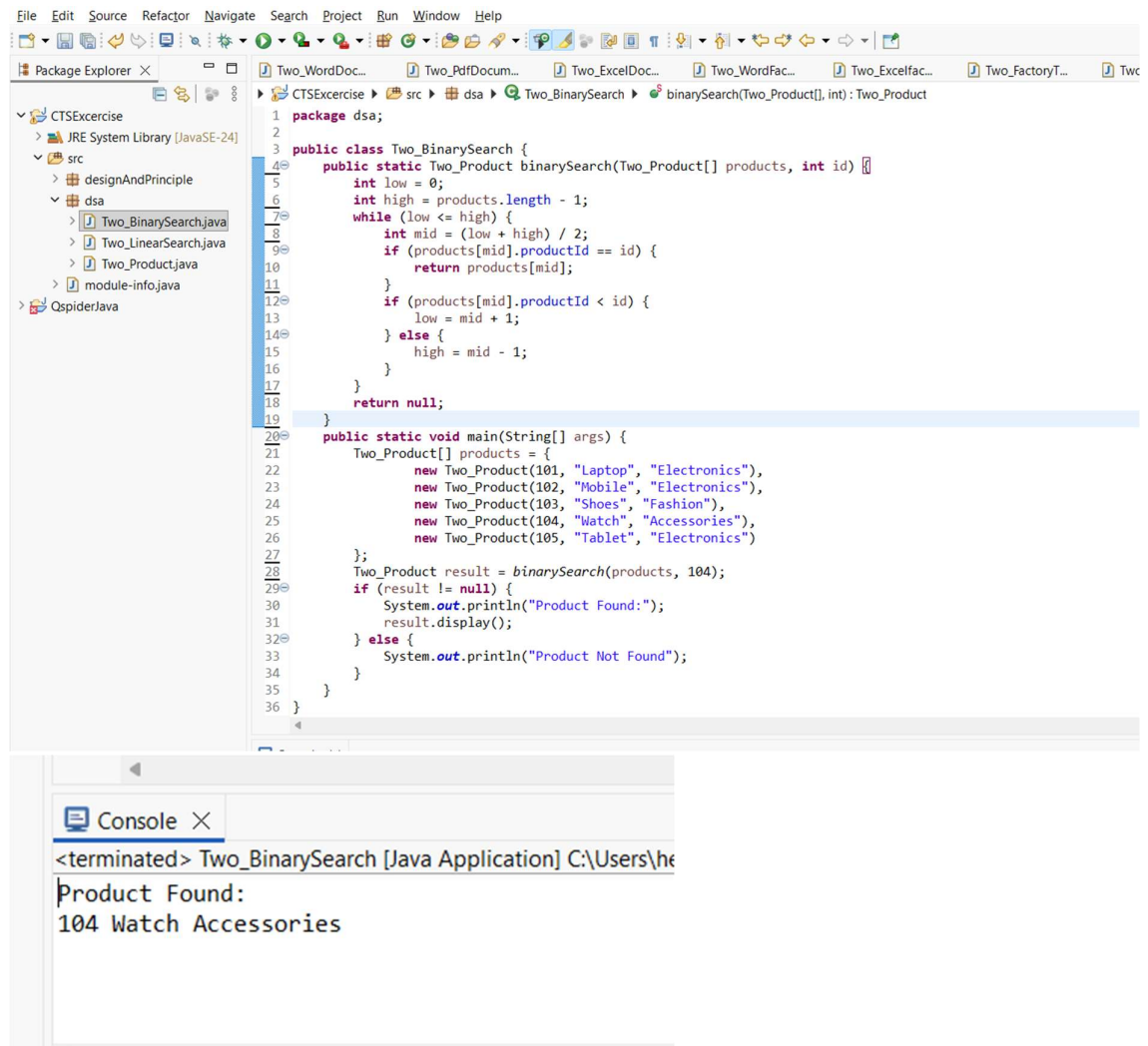
Exercise 2: E-commerce Platform Search Function

- Big O notation is used to describe the performance or efficiency of an algorithm as the input size grows.
- Search Operation is of 2 type Linear Search and binary search
- Linear search is searching the whole structure one by one with Best Case as element found at first position.--> $O(1)$ and average Case as Element found in middle.--> $O(n)$ and Worst Case as Element found at last position or not found.--> $O(n)$
- Binary Search is searching done by dividing the structure to half and searching the first half if element is lesser than mid element and the last half if mid is greater than mid element again by dividing the structure.
- Best Case is element found at middle position.--> $O(1)$ and average Case is Search space halves repeatedly.--> $O(\log n)$ and Worst Case is Element not found or found after several divisions.--> $O(\log n)$



```
1 package dsa;
2
3 public class Two_Product {
4     int productId;
5     String productName;
6     String category;
7
8     public Two_Product(int productId, String productName, String category) {
9         this.productId = productId;
10        this.productName = productName;
11        this.category = category;
12    }
13
14    public void display() {
15        System.out.println(
16            productId + " " +
17            productName + " " +
18            category);
19    }
20 }
21
```

```
1 package dsa;
2
3 public class Two_LinearSearch {
4     public static Two_Product linearSearch(Two_Product[] products, int id) {
5         for (Two_Product product : products) {
6             if (product.productId == id) {
7                 return product;
8             }
9         }
10        return null;
11    }
12    public static void main(String[] args) {
13        Two_Product[] products = {
14            new Two_Product(101, "Laptop", "Electronics"),
15            new Two_Product(102, "Mobile", "Electronics"),
16            new Two_Product(103, "Shoes", "Fashion"),
17            new Two_Product(104, "Watch", "Accessories")
18        };
19        Two_Product result = linearSearch(products, 103);
20        if (result != null) {
21            System.out.println("Product Found:");
22            result.display();
23        } else {
24            System.out.println("Product Not Found");
25        }
26    }
27 }
```



Exercise 7: Financial Forecasting

Recursion is a programming technique in which a method calls itself to solve a problem.

```
1 package dsa;
2
3 public class Seven_FinancialForecast {
4     public static double predictFutureValue(
5         double currentValue,
6         double growthRate,
7         int years) {
8         if (years == 0) {
9             return currentValue;
10        }
11        return predictFutureValue(
12            currentValue * (1 + growthRate),
13            growthRate,
14            years - 1);
15    }
16
17    public static void main(String[] args) {
18        double presentValue = 10000;
19        double growthRate = 0.10;
20        int years = 3;
21        double futureValue =
22            predictFutureValue(
23                presentValue,
24                growthRate,
25                years);
26        System.out.println(
27            "Future Value = " + futureValue);
28    }
29 }
30 }
31 }
```

Console Output:

```
<terminated> Seven_FinancialForecast [Java Application] C:\Users\hemap\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_25.0.
Future Value = 13310.000000000004
```

The recursive solution can be optimized using memoization or by using an iterative approach. The iterative method is preferred for large inputs because it uses less memory and avoids recursion overhead.